

Documentación Técnica

Módulo IA de Autorizaciones

Extracción automática de datos desde documentos de autorización de cirugía mediante Vision-Language Models

RESUMEN

Este documento detalla la implementación actual del módulo de IA para extracción automática de datos desde autorizaciones de ART / obras sociales. Cubre arquitectura, flujo completo, schemas, prompt VLM, mapeos, manejo de errores, costos, estrategia de reemplazo de proveedor, y roadmap de evolución.

Proyecto: OSSUM COR — ERP Vertical de Instrumentación Quirúrgica

Módulo: IA de Autorizaciones (VLM + Regex Fallback)

Stack: Next.js 16 · TypeScript · z-ai-web-dev-sdk · Zod

Fecha: Junio 2026

Estado: Operativo en Sandbox

Contenido

1. Resumen Ejecutivo	3
2. Arquitectura Actual	4
3. Flujo Completo	5
3.1 Selecccion del archivo	5
3.2 Envio al backend	5
3.3 Procesamiento en el backend	6
3.4 Recepcion y actualizacion del formulario	6
4. Archivos Involucrados	6
5. Variables de Entorno	7
6. Prompt Utilizado	8
7. JSON de Salida	9
7.1 Schema AutorizacionAIResponse (respuesta completa)	9
7.2 Schema AutorizacionExtracted (datos extraidos)	9
7.3 Schema MaterialAutorizadoItem (item de material)	10
8. Mapeo al Wizard	10
9. Manejo de Errores	11
10. Costos	12
10.1 Estimacion de tokens por documento	12
10.2 Costos estimados (referencia GPT-4o)	13
11. Estrategia de Reemplazo	13
11.1 Reemplazo: z-ai-vlm a OpenAI	13
11.2 Reemplazo: OpenAI a Claude (Anthropic)	14
11.3 Reemplazo: Claude a otro proveedor	14
11.4 Patron de diseno: Strategy Pattern	14
12. Checklist de Reconstruccion	15

13. Lecciones Aprendidas	16
13.1 Que funciono	16
13.2 Que no funciono	17
13.3 Errores encontrados	17
13.4 Decisiones descartadas	17
13.5 Recomendaciones futuras	18
14. Roadmap	18
14.1 Estado actual (Funcional)	18
14.2 Proxima evolucion: Factura proveedor	19
14.3 Evolucion posterior: Remitos y stock	19
14.4 Futuro: OCR Inbox universal	19

1. Resumen Ejecutivo

El modulo IA de Autorizaciones de OSSUM COR resuelve un problema critico en la operativa diaria de instrumentacion quirurgica: la carga manual de datos de autorizaciones de cirugia emitidas por obras sociales y ART (Aseguradoras de Riesgos del Trabajo). En la practica, cada autorizacion llega como un PDF o imagen escaneada con formatos inconsistentes entre prestadores, lo que obliga al personal administrativo a transcribir manualmente datos de pacientes, numeros de autorizacion, siniestros, polizas, medicos, instituciones y listados de material autorizado. Este proceso es lento, propenso a errores y no escala.

Objetivo del modulo: Automatizar la extraccion de datos desde documentos de autorizacion mediante un modelo de Vision-Language (VLM), precargando el formulario de Nueva Cirugia con los campos extraidos y dejando siempre la validacion humana como paso final antes de guardar. El sistema no reemplaza al operador: lo asiste, reduciendo el tiempo de carga de minutos a segundos y eliminando errores de tipeo.

Por que IA especializada por contexto:

- Las autorizaciones de cirugia ortopedica tienen un dominio semi-estructurado predecible: paciente, DNI, obra social, numero de autorizacion, siniestro, poliza, medico, institucion, material autorizado con codigos y precios. Un VLM entrenado para este contexto puede identificar estos campos con alta precision porque "sabe" donde buscar y que formato esperar.
- Un OCR generico (Tesseract, Google Vision OCR) devolveria texto plano sin estructura, requiriendo luego un parser de reglas por cada formato de prestador. Con cientos de obras sociales en Argentina, cada una con su formato, el mantenimiento seria inviable.
- El enfoque VLM permite procesar documentos nunca vistos sin configuracion previa, siempre que el prompt sea lo suficientemente especifico sobre el dominio ortopedico-quirurgico.

Por que se descarto OCR universal en esta etapa:

- OCR universal (texto plano + reglas) requiere mantener un diccionario de formatos por prestador. Argentina tiene mas de 300 obras sociales con formatos no estandarizados. Cada nuevo prestador implicaria codificar nuevas reglas.
- OCR no maneja bien la variabilidad visual (logos, sellos, marcas de agua, tablas con bordes incompletos) que son comunes en autorizaciones escaneadas.
- El costo del VLM por documento es bajo (ver seccion de costos) y justifica la eliminacion del mantenimiento de reglas. OCR universal se reevaluara como capa base para el OCR Inbox universal en una fase futura (ver Roadmap).

2. Arquitectura Actual

El modulo funciona como una pipeline de procesamiento que conecta la interfaz de usuario (wizard de Nueva Cirugia) con el servicio de IA (VLM via z-ai-web-dev-sdk) y devuelve los datos extraidos al formulario para validacion humana. A continuacion se detalla el flujo completo en formato de diagrama textual:

```
Usuario
|
v
Wizard Nueva Cirugia (Tab en page.tsx)
|
v
Boton / Zona de Carga de Autorizacion (drag & drop o file picker)
|
v
processFile(file) --> FormData { file, mode }
|
v
POST /api/lab/ai/autorizacion
|
v
extractAutorizacion(buffer, mimeType, mode)
|
+-- mode='mock' --> Datos hardcoded (MOCK_EXTRACTED)
|
+-- mode='vlm' --> extractWithVLM(base64, mimeType)
| |
| v
| z-ai-web-dev-sdk --> createVision()
| |
| v
| JSON parseado + validacion Zod
| |
| +-- confidence > 0.3 --> Resultado VLM
| +-- confidence <= 0.3 --> Regex Fallback
|
v
AutorizacionAIResponse (JSON estructurado)
|
```

```

v
setAiResult(data) --> Panel de Resultados IA
|
v
applyAiToForm() --> Mapeo campo a campo
|
v
SurgeryFormData (formulario precargado)
|
v
Validacion humana (edicion manual)
|
v
saveDraft() --> POST /api/lab/ocr/surgeries
|
v
Borrador guardado (in-memory store)

```

La arquitectura sigue un patron request-response clasico: el cliente sube el archivo, el backend procesa con IA y devuelve JSON estructurado, y el frontend mapea los campos al formulario. No hay WebSockets ni procesamiento asincrono en la implementacion actual. El modo mock permite desarrollo y testing sin consumir credito de IA.

3. Flujo Completo

A continuacion se documenta paso a paso que sucede desde que el usuario selecciona un archivo hasta que el formulario queda precargado. Cada paso incluye el archivo, la funcion y el formato de datos involucrado.

3.1 Seleccion del archivo

El usuario accede al tab "Nueva Cirugia" en la pagina principal. La zona de carga acepta archivos via drag-and-drop o click en el area de upload. Los formatos permitidos son PDF, JPG, JPEG, PNG, WebP y BMP, con un limite de 20 MB. Cuando el usuario suelta o selecciona un archivo, se ejecuta la funcion **processFile(file: File)** del componente page.tsx. Esta funcion configura el estado de procesamiento (isProcessing=true, barra de progreso simulada) y muestra el panel de resultados IA.

3.2 Envio al backend

La funcion `processFile` construye un objeto `FormData` con dos campos: **file** (el archivo binario) y **mode** (string: "mock" o "vlm", segun el toggle del header). Se ejecuta un fetch POST a `/api/lab/ai/autorizacion`. Mientras espera la respuesta, se simula una barra de progreso que incrementa aleatoriamente cada 400ms hasta un maximo del 90%.

3.3 Procesamiento en el backend

El endpoint POST `/api/lab/ai/autorizacion` recibe el `FormData`, valida tipo y tamaño del archivo, convierte el File a Buffer y llama a `extractAutorizacion(buffer, mimeType, mode)`. Si `mode="mock"`, retorna inmediatamente datos hardcoded. Si `mode="vlm"`, convierte el buffer a base64, construye un data URI (`data:mimeType;base64,...`) y llama a `extractWithVLM()`, que utiliza `z-ai-web-dev-sdk` para invocar el modelo de vision. La respuesta del modelo se parsea como JSON y se valida con el schema `Zod AutorizacionAIResponseSchema` antes de retornar al cliente.

3.4 Recepcion y actualizacion del formulario

El cliente recibe el JSON `AutorizacionAIResponse` y lo almacena en el estado `aiResult`. El panel de resultados muestra: indicador de si parece una autorizacion (`looks_like_authorization`), nivel de confianza (`confidence`), proveedor utilizado (`z-ai-vlm / mock / fallback-regex`), advertencias y un resumen de los datos extraidos. El usuario hace click en "Aplicar al formulario" que ejecuta **`applyAiToForm()`**, la cual mapea campo por campo desde `aiResult.extracted` al estado `SurgeryFormData`. El DNI se normaliza automaticamente (quita puntos y guiones). El formulario queda precargado pero editable, y el usuario puede corregir cualquier campo antes de guardar.

4. Archivos Involucrados

La siguiente tabla documenta cada archivo del modulo con su funcion especifica dentro de la arquitectura. Todos los archivos estan bajo `/home/z/my-project/` que es la raiz del proyecto.

Archivo	Funcion
<code>src/app/page.tsx</code>	Componente principal UI. Contiene el wizard Nueva Cirugia (Tab 4), la zona de upload, el panel de resultados IA, el formulario de cirugia con todos los campos, y la barra de acciones (Guardar Borrador / Limpiar).
<code>src/lib/services/ai/autorizacion-extractor.ts</code>	Servicio central de extraccion. Contiene <code>extractAutorizacion()</code> (entry point), <code>extractWithVLM()</code> (llamada al modelo de vision via <code>z-ai-web-dev-sdk</code>), <code>extractWithRegex()</code> (fallback con patrones regex), y <code>MOCK_EXTRACTED</code> (datos de prueba).
<code>src/lib/validators/autorizacion-ai.ts</code>	Schemas Zod y tipos TypeScript. Define <code>MaterialAutorizadoItemSchema</code> , <code>AutorizacionExtractedSchema</code> , <code>AutorizacionAIResponseSchema</code> , y funciones auxiliares: <code>normalizeDni()</code> , <code>normalizeDate()</code> , <code>emptyAutorizacionResponse()</code> .

src/app/api/lab/ai/autorizacion/route.ts	Endpoint API POST. Valida tipo/tamano del archivo, convierte a Buffer, invoca extractAutorizacion(), valida la respuesta con Zod, y retorna JSON.
src/app/api/lab/ocr/surgeries/route.ts	Endpoint API para guardar borradores. POST: guarda SurgeryFormData en store in-memory. GET: lista todas las cirugías guardadas. No persiste en base de datos.
src/app/layout.tsx	Layout raíz de Next.js. Define fuentes (Geist, Geist Mono), metadata del sitio, y envuelve con Toaster.
src/lib/db.ts	Cliente Prisma singleton. Configurado para SQLite. No utilizado actualmente por el modulo de autorizaciones.
prisma/schema.prisma	Schema de base de datos. Solo tiene modelos User y Post. No existe modelo Surgery ni Autorizacion.
.env	Variables de entorno. Solo contiene DATABASE_URL para SQLite. No hay API keys de IA (z-ai-web-dev-sdk las descubre automaticamente).
package.json	Dependencias clave: z-ai-web-dev-sdk (^0.0.17), zod (^4.0.2), sharp (^0.34.3, no utilizado aun), next (^16.1.1), react (^19.0.0).

Tabla 1: Archivos del modulo y sus funciones

5. Variables de Entorno

La implementacion actual utiliza z-ai-web-dev-sdk que descubre automaticamente las credenciales del entorno de ejecucion (sandbox Z.ai). No se requieren API keys explicitas en variables de entorno para el funcionamiento en sandbox. Sin embargo, para produccion o migracion a otro proveedor, las siguientes variables seran necesarias:

Variable	Estado actual	Obligatoria	Descripcion
OPENAI_API_KEY	No configurada	No (solo si se migra a OpenAI)	Clave API de OpenAI para GPT-4 Vision o modelos equivalentes.
GEMINI_API_KEY	No configurada	No (solo si se migra a Gemini)	Clave API de Google Gemini para modelos de vision.
AI_PROVIDER	No configurada (implicito: z-ai-vlm)	No	Identificador del proveedor de IA. Actualmente hardcodeado en el codigo.
AI_MODEL	No configurada (decide el SDK)	No	Modelo especifico a utilizar. El SDK selecciona automaticamente.
DATABASE_URL	file:/home/z/my-project/db/custom.db	Si (Prisma)	URL de conexion a base de datos SQLite. No usada por el modulo de IA actualmente.

Tabla 2: Variables de entorno del sistema

Nota critica: El proveedor de IA actual (z-ai-vlm) no requiere configuracion de API keys porque el SDK se autentica contra el entorno Z.ai automaticamente. Si se migra a OpenAI o Gemini, las claves correspondientes deberan configurarse y el codigo debera modificarse para pasarlas al cliente del nuevo SDK. Ver Seccion 11 para la estrategia de reemplazo.

6. Prompt Utilizado

El prompt enviado al modelo VLM esta definido en la funcion `extractWithVLM()` del archivo `autorizacion-extractor.ts`. Se presenta a continuacion en su version integra, sin resumir:

```
Extrae los datos de esta autorizacion de cirugia/art/obra social. Responde SOLO con JSON valido, sin markdown ni explicaciones:

{
  "paciente": "",
  "dni": "",
  "medico": "",
  "institucion": "",
  "obra_social": "",
  "numero_autorizacion": "",
  "numero_siniestro": "",
  "numero_poliza": "",
  "fecha_autorizacion": "",
  "fecha_cirugia": "",
  "material_autorizado": [{"codigo":"","descripcion":"","cantidad":"","precio_referencia":""}],
  "observaciones": "",
  "looks_like_authorization": true/false,
  "confidence": 0.0-1.0
}
```

Analisis del prompt:

- **Idioma:** El prompt esta en espanol argentino (uso de "Extrae" con acento diacritico), coincidiendo con el dominio de los documentos que se procesan (autorizaciones argentinas).
- **Instruccion de formato:** "Responde SOLO con JSON valido, sin markdown ni explicaciones" busca evitar que el modelo envuelva la respuesta en bloques de codigo markdown (``json ... ``). Sin embargo, el codigo de parsing usa un regex que extrae el primer {...} del texto, lo que funciona incluso si el modelo ignora la instruccion.
- **Schema embebido:** El JSON schema se incluye directamente en el prompt como template, con campos vacios. Esto es mas efectivo que describir el schema en texto porque el modelo puede

copiar la estructura directamente.

- **Campo looks_like_authorization:** Permite al modelo indicar si el documento parece realmente una autorizacion. Si devuelve false, la UI muestra una advertencia. El default en el parsing es true si el campo falta.
- **Campo confidence:** El modelo evalua su propia confianza en la extraccion. Si es menor a 0.3, el sistema activa el fallback a regex. El default en el parsing es 0.7.
- **material_autorizado como array:** El prompt solicita una lista de items con codigo, descripcion, cantidad y precio_referencia. Esto es critico para el caso de uso principal de instrumentacion quirurgica donde cada cirugia tiene multiples implantes autorizados.

7. JSON de Salida

El esquema exacto del JSON de salida esta definido por los schemas Zod en autorizacion-ai.ts. A continuacion se documenta cada campo con su tipo, obligatoriedad y descripcion.

7.1 Schema AutorizacionAIResponse (respuesta completa)

Campo	Tipo	Obligatorio	Descripcion
provider	enum: 'z-ai-vlm' 'fallback-regex' 'mock'	Si	Identifica que motor genero la respuesta.
confidence	number (0.0 - 1.0)	Si (default: 0)	Nivel de confianza de la extraccion. Umbral de fallback: 0.3.
looks_like_authorization	boolean	Si (default: false)	Si el documento parece una autorizacion de cirugía.
warnings	string[]	Si (default: [])	Lista de advertencias sobre el procesamiento.
extracted	AutorizacionExtracted	Si	Objeto con los datos extraidos del documento.
raw_text_preview	string	Si (default: "")	Vista previa del texto crudo devuelto por el modelo (max 500 chars).

Tabla 3: Schema AutorizacionAIResponse

7.2 Schema AutorizacionExtracted (datos extraidos)

Campo	Tipo	Obligatorio	Descripcion
paciente	string	Si (default: "")	Nombre completo del paciente.

dni	string	Si (default: "")	DNI del paciente (sin normalizar).
medico	string	Si (default: "")	Nombre del medico solicitante.
institucion	string	Si (default: "")	Nombre del sanatorio/hospital/clinica.
obra_social	string	Si (default: "")	Obra social, ART o prepaga.
numero_autorizacion	string	Si (default: "")	Numero de autorizacion de la cirugía.
numero_siniestro	string	Si (default: "")	Numero de siniestro (para ART).
numero_poliza	string	Si (default: "")	Numero de poliza del asegurado.
fecha_autorizacion	string	Si (default: "")	Fecha de emision de la autorizacion.
fecha_cirugia	string	Si (default: "")	Fecha programada de la cirugía.
material_autorizado	MaterialAutorizadoItem[]	Si (default: [])	Lista de materiales/implantes autorizados.
observaciones	string	Si (default: "")	Observaciones generales del documento.

Tabla 4: Schema AutorizacionExtracted

7.3 Schema MaterialAutorizadoItem (item de material)

Campo	Tipo	Obligatorio	Descripcion
codigo	string	Si (default: "")	Codigo del implante/material (ej: IMPL-TQ-001).
descripcion	string	Si (default: "")	Descripcion del material (ej: Tornillo canulado titanio 6.5mm x 50mm).
cantidad	string	Si (default: "")	Cantidad autorizada. String (no number) para permitir "4", "2-3", etc.
precio_referencia	string	Si (default: "")	Precio de referencia. String para mantener formato original (ej: "\$85.000").

Tabla 5: Schema MaterialAutorizadoItem

8. Mapeo al Wizard

La funcion `applyAiToForm()` en `page.tsx` realiza el mapeo campo a campo desde el objeto `AutorizacionExtracted` (devuelto por la IA) al estado `SurgeryFormData` (que alimenta el formulario). A continuacion se documenta cada mapeo con las transformaciones aplicadas:

Campo JSON (AutorizacionExtracted)	Campo Formulario (SurgeryFormData)	Transformacion
------------------------------------	------------------------------------	----------------

paciente	patientName	Sin transformacion. Se asigna directo.
dni	patientDni	normalizeDni(): quita puntos, guiones y caracteres no numericos.
medico	doctorName	Sin transformacion.
institucion	institution	Sin transformacion.
obra_social	obraSocial	Sin transformacion.
numero_autorizacion	numeroAutorizacion	Sin transformacion.
numero_siniestro	numeroSiniestro	Sin transformacion.
numero_poliza	numeroPoliza	Sin transformacion.
fecha_autorizacion	fechaAutorizacion	Sin transformacion (BUG: no usa normalizeDate()). El campo input[type=date] requiere YYYY-MM-DD.
fecha_cirugia	fechaCirugia	Sin transformacion (BUG: no usa normalizeDate()). El campo input[type=date] requiere YYYY-MM-DD.
material_autorizado	materialAutorizado	Si el array viene vacio, se inserta un item vacio como placeholder.
observaciones	observaciones	Sin transformacion.

Tabla 6: Mapeo de campos IA a formulario

Bug conocido: La funcion normalizeDate() existe en autorizacion-ai.ts y convierte formatos DD/MM/YYYY a YYYY-MM-DD, pero applyAiToForm() no la utiliza para los campos fechaAutorizacion y fechaCirugia. Esto significa que si la IA devuelve una fecha en formato DD/MM/YYYY, el campo input[type=date] del formulario no la renderizara correctamente porque ese tipo de input solo acepta YYYY-MM-DD. Este bug debe corregirse en la proxima iteracion.

9. Manejo de Errores

El sistema implementa una estrategia de defensa en profundidad con multiples capas de fallback y manejo granular de errores. A continuacion se documenta cada tipo de error posible y como se maneja en la implementacion actual.

Tipo de error	Donde se detecta	Manejo actual	Gap / Mejora pendiente
PDF invalido o corrupto	API route (lectura de FormData)	El endpoint convierte el File a Buffer sin validar que sea un PDF valido. Si el VLM no puede procesarlo, devuelve confianza baja y activa regex fallback.	Se deberia validar la estructura del PDF antes de enviar al VLM.

Imagen corrupta	API route	Idem PDF. El VLM recibira bytes invalidos y probablemente devolvera una respuesta vacia o con baja confianza.	Validar que la imagen se pueda decodificar (con sharp) antes de enviar al VLM.
Timeout de IA	extractWithVLM()	No hay timeout explicito. Si el SDK tarda demasiado, el request del cliente eventualmente falla con un error generico. El catch en processFile() muestra el mensaje de error.	Implementar AbortController con timeout configurable (ej: 30 segundos).
Respuesta incompleta de IA	extractWithVLM()	Si el JSON devuelto no tiene todos los campos, los campos faltantes se rellenan con "" (gracias a los defaults de Zod). El formulario se carga con campos vacios donde la IA no extrajo datos.	Mostrar visualmente cuales campos fueron extraidos vs. cuales estan vacios.
Campos faltantes	applyAiToForm()	Los defaults de Zod garantizan que todos los campos existen (como strings vacios). El usuario ve campos en blanco y puede completarlos manualmente.	Agregar indicadores visuales de "campo extraido por IA" vs "campo completado manualmente".
Proveedor IA caido	extractWithVLM()	El catch en extractWithVLM() captura cualquier error del SDK y devuelve emptyAutorizacionResponse('fallback-regex') con un warning. Sin embargo, el regex fallback no funciona porque no hay OCR previo (el texto es un placeholder).	El regex fallback es teoricamente correcto pero practicamente no funcional sin una capa de OCR. Se debe agregar OCR (Tesseract/sharp) como paso previo al regex.
JSON malformado del VLM	extractWithVLM()	Se usa regex (match de {...}) para extraer JSON del texto. Si falla el parseo, se devuelve emptyAutorizacionResponse con warning.	Considerar usar JSON.parse con try/catch y fallback a regex de extraccion mas robusto.

Tabla 7: Manejo de errores por tipo

10. Costos

Los costos se estiman en base al consumo tipico de tokens para el procesamiento de una autorizacion de cirugia. Los valores son aproximados y dependen del proveedor de IA y el modelo utilizado. Los precios se calculan sobre la base del modelo GPT-4o (OpenAI) como referencia de mercado, ya que el modelo exacto utilizado por z-ai-web-dev-sdk no es publico.

10.1 Estimacion de tokens por documento

Componente	Tokens estimados	Notas
------------	------------------	-------

Prompt (texto)	~200-250 tokens	Instruccion + schema JSON de ejemplo.
Imagen (autorizacion)	~1000-2000 tokens	Depende de la resolucion. Una autorizacion tipica es 1 pagina A4 escaneada.
Respuesta JSON	~150-250 tokens	12 campos + 1-5 items de material.
Total por documento	~1350-2500 tokens	Promedio: ~1800 tokens.

Tabla 8: Estimacion de tokens por documento

10.2 Costos estimados (referencia GPT-4o)

Volumen	Tokens totales	Costo estimado (USD)	Notas
1 documento	~1,800	\$0.005 - \$0.015	Depende de si la imagen se cobra como input o vision.
100 documentos	~180,000	\$0.50 - \$1.50	Aproximadamente 1 semana de operacion en un centro medico chico.
1,000 documentos	~1,800,000	\$5.00 - \$15.00	Aproximadamente 3 meses de operacion.
10,000 documentos	~18,000,000	\$50.00 - \$150.00	Volumen anual de un centro medico mediano.

Tabla 9: Estimacion de costos por volumen

Nota importante: Estos costos son estimaciones basadas en precios publicos de OpenAI. El modelo utilizado por z-ai-web-dev-sdk en el sandbox de Z.ai puede tener costos diferentes o ser gratuito en el contexto del sandbox. En produccion, se debera verificar el costo real con el proveedor seleccionado. Con modelos mas economicos como GPT-4o-mini o Claude Haiku, los costos podrian reducirse en un 80-90%.

11. Estrategia de Reemplazo

El sistema esta disenado para permitir el reemplazo del proveedor de IA sin reescribir la logica de negocio. La clave esta en que la funcion `extractAutorizacion()` es el unico punto de contacto con el modelo de IA. Todo lo demas (schemas, validacion, mapeo, UI) es agnostico del proveedor. A continuacion se describe como cambiar de proveedor.

11.1 Reemplazo: z-ai-vlm a OpenAI

Paso 1: Crear un adaptador OpenAI en `src/lib/services/ai/providers/openai.ts` que implemente la misma interfaz que `extractWithVLM()`. Este adaptador usaria el SDK oficial de OpenAI (openai npm package) para llamar a gpt-4o con el mismo prompt y la imagen en base64. La respuesta se parsea de la misma

manera (regex para extraer JSON).

Paso 2: Agregar la variable de entorno OPENAI_API_KEY al archivo .env.local. Modificar extractAutorizacion() para seleccionar el proveedor segun una variable de entorno o parametro de configuracion. El campo provider en la respuesta cambiaria de "z-ai-vlm" a "openai".

Paso 3: Actualizar el enum provider en el schema Zod para incluir "openai". No se requiere ningun cambio en el frontend ni en el mapeo de campos.

11.2 Reemplazo: OpenAI a Claude (Anthropic)

El proceso es analogo. Se crea un adaptador en src/lib/services/ai/providers/anthropic.ts que use el SDK @anthropic-ai/sdk. Claude soporta mensajes multimodales con el mismo formato (texto + imagen base64). El prompt se reutiliza sin cambios. La variable de entorno seria ANTHROPIC_API_KEY. Se agrega "anthropic" al enum provider en Zod.

11.3 Reemplazo: Claude a otro proveedor

Cualquier proveedor que soporte mensajes multimodales (texto + imagen) puede integrarse siguiendo el mismo patron: crear un adaptador, configurar API key, agregar al enum provider. La interfaz del adaptador es simple: recibe base64Image y mimeType, devuelve AutorizacionAIResponse. Si el proveedor no soporta vision directamente, se puede agregar una capa de OCR previa (Tesseract, Google Vision) para extraer texto y luego pasar el texto a un modelo de lenguaje standard (GPT-3.5, Claude Haiku).

11.4 Patron de diseno: Strategy Pattern

La arquitectura ideal para multi-proveedor seria implementar un Strategy Pattern donde extractAutorizacion() delega a un proveedor seleccionado por configuracion:

```
interface AIProvider {
  extract(base64Image: string, mimeType: string): Promise<AutorizacionAIResponse>;
}

class ZAIVLMPProvider implements AIProvider { ... }
class OpenAIProvider implements AIProvider { ... }
class AnthropicProvider implements AIProvider { ... }
class MockProvider implements AIProvider { ... }

// Seleccion por variable de entorno
const provider = process.env.AI_PROVIDER || 'z-ai-vlm';
const extractor = PROVIDERS[provider];
```

Este patron permite agregar proveedores sin modificar la logica de negocio y cambiar el proveedor activo con una sola variable de entorno. Es la evolucion recomendada para produccion.

12. Checklist de Reconstruccion

La siguiente checklist permite a cualquier desarrollador reconstruir el modulo desde cero en un proyecto nuevo. Cada paso esta ordenado por dependencia y puede verificarse independientemente.

PASO 1

Crear proyecto Next.js 16 con TypeScript, Tailwind CSS 4, y shadcn/ui. Verificar que el proyecto compila y corre con `npm run dev`.

PASO 2

Instalar dependencias: zod, z-ai-web-dev-sdk, sharp (opcional). Ejecutar: `npm install zod z-ai-web-dev-sdk sharp`.

PASO 3

Crear el directorio `src/lib/validators/` y el archivo `autorizacion-ai.ts`. Definir los schemas Zod (`MaterialAutorizadoItemSchema`, `AutorizacionExtractedSchema`, `AutorizacionAIResponseSchema`) y las funciones auxiliares (`normalizeDni`, `normalizeDate`, `emptyAutorizacionResponse`). Verificar con un test unitario que los schemas parsean correctamente un JSON de ejemplo.

PASO 4

Crear el directorio `src/lib/services/ai/` y el archivo `autorizacion-extractor.ts`. Implementar `extractWithRegex()` con los patrones regex para documentos argentinos. Verificar con un test que extrae correctamente de un texto de ejemplo.

PASO 5

Implementar `extractWithVLM()` en el mismo archivo. Utilizar `z-ai-web-dev-sdk` para llamar a `createVision()` con el prompt documentado en la Seccion 6. Implementar el parsing de respuesta (regex para extraer JSON, parsear, mapear a `AutorizacionExtracted`). Verificar con un archivo de imagen real.

PASO 6

Implementar `extractAutorizacion()` como funcion orquestadora. Soportar `mode="mock"` (datos hardcoded) y `mode="vlm"` (con fallback a regex). Verificar que el mock funciona y que el fallback se activa cuando la confianza es menor a 0.3.

PASO 7

Crear el endpoint API POST `/api/lab/ai/autorizacion/route.ts`. Validar tipo de archivo (`ALLOWED_TYPES`) y tamaño (20MB max). Convertir File a Buffer, llamar `extractAutorizacion()`, validar respuesta con Zod schema, retornar JSON. Probar con curl.

PASO 8

Crear el endpoint API POST `/api/lab/ocr/surgeries/route.ts` para guardar borradores. Implementar store in-memory como solucion temporaria. Generar numero de cirugia automatico.

PASO 9

Construir la interfaz de usuario en `page.tsx`. Implementar el tab "Nueva Cirugia" con: zona de upload (drag-and-drop + file picker), panel de resultados IA, formulario con 12 campos + tabla dinamica de material, y barra de acciones. Implementar `processFile()`, `applyAiToForm()`, `saveDraft()`, y los handlers de drag-and-drop.

PASO 10

Agregar el toggle Mock/VLM en el header. Enlazar el estado `useMock` al parametro `mode` del `FormData` en `processFile()`. Verificar que el toggle cambia entre datos mock y procesamiento real.

PASO 11

Probar el flujo completo end-to-end: subir un PDF o imagen, verificar que la IA extrae datos, aplicar al formulario, editar campos, guardar como borrador. Verificar que el endpoint de borradores retorna el dato guardado.

PASO 12

Ejecutar `npm run build` para verificar que el proyecto compila sin errores en modo produccion. Corregir cualquier error de TypeScript o build.

13. Lecciones Aprendidas

13.1 Que funciono

- **Prompt especifico por dominio:** Enviar el schema JSON completo como parte del prompt produjo respuestas mucho mas consistentes que describir los campos en texto. El modelo tiende a respetar la estructura proporcionada y completa los campos con datos reales del documento en lugar de inventar respuestas.
- **Modo mock para desarrollo:** Tener datos hardcoded permitio desarrollar y probar toda la UI y el flujo de integracion sin depender del servicio de IA. Esto acelero el desarrollo significativamente y elimino la friccion de esperar respuestas del modelo durante el desarrollo.
- **Validacion Zod como contrato:** Definir los schemas Zod primero y usarlos tanto en el backend (validacion de respuesta) como en el frontend (tipos TypeScript derivados) creo un contrato tipado que evito errores de integracion. Cualquier cambio en el schema se propaga automaticamente a ambos lados.
- **VLM en lugar de OCR puro:** El enfoque de Vision-Language Modelo evito la necesidad de mantener reglas de parsing por formato de prestador. Un solo prompt funciona para cualquier

formato de autorizacion, lo que es critico en el mercado argentino con cientos de obras sociales.

- **Barra de progreso simulada:** Aunque la barra no refleja el progreso real del backend, mejora la experiencia de usuario al dar feedback visual inmediato mientras espera la respuesta de la IA.

13.2 Que no funciona

- **Regex fallback sin OCR:** El fallback a regex fue diseñado para funcionar sobre texto extraído previamente por OCR. Sin embargo, no se implementó la capa de OCR (Tesseract o similar), por lo que el regex opera sobre un placeholder string y nunca extrae datos reales. Es funcionalmente inútil en la implementación actual.
- **Procesamiento de PDFs:** El código convierte PDFs a base64 y los envía directamente al VLM como si fueran imágenes. Esto funciona para PDFs de una sola página con escaneos, pero falla con PDFs multi-página o con contenido vectorial. La librería sharp está instalada pero no se utiliza para convertir páginas de PDF a imágenes.
- **Normalización de fechas:** Se implementó `normalizeDate()` pero no se integra en `applyAiToForm()`. Las fechas extraídas por la IA en formato DD/MM/YYYY no se convierten a YYYY-MM-DD, lo que causa que los campos `input[type=date]` no las rendericen.
- **Persistencia:** Los borradores se guardan en un objeto JavaScript in-memory que se pierde al reiniciar el servidor. No existe un modelo Surgery en Prisma. Esto es aceptable para sandbox pero bloqueante para producción.

13.3 Errores encontrados

- **VLM devuelve JSON en markdown:** Algunas veces el modelo ignora la instrucción de "SOLO JSON" y envuelve la respuesta en ````json ... ````. El regex de parsing (`match de {...}`) lo maneja, pero sería más robusto strippear markdown antes de parsear.
- **Confidence inconsistente:** El campo confidence devuelto por el VLM no siempre es calibrado. A veces devuelve 0.9 para extracciones imperfectas y 0.5 para extracciones correctas. No se debería confiar ciegamente en este valor para automatizar decisiones.
- **Material_autorizado vacio:** En documentos donde el listado de material está en una tabla compleja con bordes, el VLM a veces no extrae los items individuales y devuelve `material_autorizado` como array vacío. Se necesitarían prompts más específicos para tablas.

13.4 Decisiones descartadas

- **OCR puro (Tesseract) + NER:** Se evaluó usar OCR para extraer texto y luego un modelo NER (Named Entity Recognition) para identificar campos. Se descartó por la alta variabilidad de formatos entre prestadores argentinos y la necesidad de entrenar un NER para el dominio.

- **Plantillas por prestador:** Se considero crear templates de extraccion para cada obra social. Se descarto por la cantidad de prestadores (300+) y la velocidad a la que cambian sus formatos.
- **Procesamiento async con cola:** Se considero usar una cola (Bull/BullMQ) para procesar documentos en background. Se descarto por la complejidad innecesaria para el volumen actual y la latencia aceptable del VLM (2-5 segundos por documento).

13.5 Recomendaciones futuras

- **Implementar OCR como capa base:** Agregar Tesseract o Google Vision OCR como paso previo al regex fallback. Esto haria funcional el camino de fallback cuando el VLM no esta disponible.
- **Convertir PDFs a imagenes:** Usar sharp o pdf2pic para convertir cada pagina de PDF a imagen antes de enviar al VLM. Esto garantiza compatibilidad con PDFs de cualquier tipo.
- **Implementar Strategy Pattern para proveedores:** Abstraer el proveedor de IA detras de una interfaz comun para facilitar el cambio y testing con mocks mas sofisticados.
- **Agregar metricas de calidad:** Registrar confianza por campo, no solo global. Permitir al usuario calificar la extraccion para alimentar un dataset de fine-tuning futuro.
- **Persistir borradores en base de datos:** Crear modelo Surgery en Prisma y migrar el store in-memory a SQLite/PostgreSQL.

14. Roadmap

14.1 Estado actual (Funcional)

El modulo actualmente funciona de punta a punta en modo sandbox. Las funcionalidades implementadas y operativas son las siguientes:

- Extraccion de datos desde imagenes (JPG/PNG/WebP/BMP) via VLM con z-ai-web-dev-sdk.
- Extraccion desde PDFs (funcional para PDFs de 1 pagina con contenido escaneado).
- Modo mock con datos de prueba para desarrollo sin consumo de IA.
- Formulario completo de Nueva Cirugia con 12 campos + tabla dinamica de material.
- Mapeo automatico IA a formulario con normalizacion de DNI.
- Panel de resultados IA con indicadores de confianza y advertencias.
- Guardado de borradores en store in-memory (no persistente).
- Toggle Mock/VLM en la UI para cambiar modo de procesamiento.

14.2 Proxima evolucion: Factura proveedor

La proxima iteracion del modulo debe soportar la extraccion de datos desde facturas de proveedores de implantes. Las facturas tienen un formato diferente a las autorizaciones: incluyen datos del proveedor (CUIT, razon social), items facturados con precios unitarios, subtotales, IVA, y total. El prompt debera ser diferente al de autorizaciones pero la arquitectura (VLM + schema Zod + mapeo a formulario) se reutiliza completamente. Se debera crear un nuevo extractor (factura-extractor.ts), un nuevo schema Zod (factura-ai.ts), y un nuevo endpoint API. La UI debera agregar un tab o flujo para "Cargar Factura Proveedor" con su propio formulario y tabla de items.

14.3 Evolucion posterior: Remitos y stock

Despues de facturas, la siguiente evolucion es procesar remitos (documentos de entrega de material). Los remitos son criticos para el control de stock: cuando un proveedor entrega implantes, el remito confirma que el material llego fisicamente. El flujo sera: cargar remito via IA, extraer items entregados, comparar con lo facturado (conciliacion automatica), y actualizar el stock disponible. Esto requiere un modelo de base de datos para inventario y la integracion con el modulo de facturas para conciliacion.

14.4 Futuro: OCR Inbox universal

La vision a largo plazo es un OCR Inbox universal que pueda recibir cualquier tipo de documento (autorizacion, factura, remito, historia clinica, pedido medico) y clasificarlo automaticamente antes de procesarlo. El flujo seria: el usuario sube cualquier documento, un clasificador de IA identifica el tipo de documento (autorizacion, factura, remito, otro), y lo enruta al extractor especializado correspondiente. Los documentos que no coincidan con ningun tipo conocido se procesarian con un OCR generico y se almacenarian para revision manual. Este OCR Inbox universal es el destino final del tab "OCR Inbox" que hoy muestra solo un placeholder vacio. El tab "Workspace" seria donde el usuario trabaja sobre un documento especifico, y el tab "Historial" mostraria todos los documentos procesados con su estado.